



POLITECNICO DI BARI

DEPARTMENT OF ELECTRICAL AND INFORMATION ENGINEERING
Master Degree in Automation Engineering

Mobile Robotics

Autonomous Leader–Follower Navigation with Visual Target Recovery

Supervisors

Prof. Eng. Luca De Cicco
Eng. Nunzio Barone

Student

Luigi Pio Ficco

Academic Year 2025 – 2026

Abstract

This documentation presents the design, implementation and experimental evaluation of a vision-based leader–follower navigation system for a TurtleBot3 robot. The project was developed entirely in simulation, using ROS2 Jazzy as the robotics framework and Gazebo Harmonic as the simulation environment.

The objective is to enable a follower robot to track and pursue a moving target, identified by an ArUco marker mounted on its rear panel, keeping a prescribed following distance while avoiding obstacles. The follower has no prior knowledge of the target’s route and relies exclusively on onboard perception: an RGB camera for detection and a 2D LiDAR for safety.

The distinguishing contribution of this work is not the pursuit itself, which is a classical problem, but the *recovery of the target after a loss of visibility*. When the marker leaves the camera field of view, the system estimates the target’s motion from the last available observations and continues the pursuit blind for a bounded time horizon; if that fails, it falls back on an active search, rotating in place until the marker reappears.

The follower deliberately uses *no map, no SLAM and no global planner*: its navigation is entirely reactive and perception-driven. The target, which is the scripted stimulus and not the object of study, is driven by the official Nav2 navigation stack (AMCL for localisation against a known map, plus a planner and a controller), configured rather than reimplemented, following a route defined once and shared with every other geometric check in the project. The design is validated quantitatively by comparing the proposed predictive strategy against a purely reactive baseline, which stops as soon as the target is lost, over repeated runs on an identical, reproducible target route.

Wherever this project reuses a substantial pattern from an existing, licensed codebase (ROBOTIS’ own TurtleBot3 packages, the official Nav2 stack, or a third-party ROS2 logging utility) or from the course’s own theoretical material, the reuse is declared explicitly, in the code and in Section 12 of this document, together with what was written original to it and why.

Contents

1	Introduction and Objectives	4
1.1	Problem statement	4
1.2	Design Philosophy	4
2	System Overview	5
2.1	Architecture	5
2.2	Topic interface	6
2.3	Software environment	6
3	Simulation Environment	6
3.1	The arena	6
3.2	The two robots	6
3.2.1	The camera was replaced	7
3.2.2	How the marker is mounted on the target	7
4	Perception: Marker Detection and Pose Estimation	7
4.1	The marker	7
4.2	From the image to the target position	7
4.3	The detector node	7
4.4	The dominant loss mechanism: incidence angle	8
5	Reference Frames	8
5.1	The velocity must be estimated in a fixed frame	8
5.2	Two things kept out of the transform tree on purpose	8
6	Pursuit Control	9
6.1	The control law	9
6.2	Where the law comes from	9
6.3	What each term buys	9
6.4	Gain tuning	10
7	Motion Prediction	10
7.1	Estimating how fast the target is moving	10
7.2	Predicting where it went	10
7.3	The same code serves both pursuit and prediction	10
7.4	The prediction horizon	11
8	The Recovery State Machine	11
8.1	States and transitions	11
8.2	Abandoning the prediction on a sensor contradiction	11
8.3	The two strategies under comparison	12
9	Safety Arbitration	12
9.1	The conflict	12
9.2	Provenance of this section's code	12
9.3	How the LiDAR limits the visual command	13
10	Experimental Infrastructure	13
10.1	The target as a reproducible stimulus	13
10.2	Why Nav2 drives the target	13
10.3	What is official Nav2 and what is this project's own	13

10.4	Two parameters worth naming explicitly	14
10.5	Two turns deliberately placed away from any obstacle	14
10.6	Ground truth for the target, and why it is legitimate	15
11	Testing and Validation	15
11.1	Experimental protocol	15
11.2	Metrics	15
11.3	Results	16
11.3.1	What the comparison shows	16
11.3.2	An honest reading of the RMSE figure	16
11.4	Summary of the work	17
11.5	Limitations	17
11.6	Future work	17
12	Provenance of the Code	18
13	Repository	19

1 Introduction and Objectives

1.1 Problem statement

Leader–follower navigation is the problem of making one mobile robot (the *follower*) maintain a prescribed spatial relationship with respect to another moving agent (the *leader*, or *target*), using only onboard sensing. It is the abstraction behind a wide range of applications: automated load carriers that follow a warehouse operator, convoys of ground vehicles in which only the first is driven, service robots that accompany a person through a building.

The hard part is not the pursuit itself, which is a classical control problem, but what happens when the target disappears. A camera has a finite field of view; a marker is readable only within a limited range and incidence angle. The target *will* vanish, repeatedly, and what the follower does in those seconds is what separates a demonstration from a usable system.

This is the behaviour the project proposal commits to:

[...] When the target leaves the camera field of view, the system estimates its most likely position from the last available observations and continues the pursuit for a limited time before switching to an active search behaviour. The final system will be evaluated by comparing a purely reactive following strategy, which stops as soon as the target is lost, against the proposed predictive strategy.

Three commitments follow from this statement and structure the whole document:

- **Perception-only operation.** The follower does not know the target’s route, and no map of the environment is used by it. Every decision is taken from the current camera and LiDAR readings plus a short memory of past detections.
- **Bounded blind pursuit.** Prediction is explicitly limited in time. An unbounded extrapolation would eventually chase a fiction; the horizon is a design parameter with a measurable trade-off, discussed in Section 7.4.
- **Quantitative comparison.** The system must be evaluated, not merely demonstrated. Two strategies, identical in every respect except the predictive stage, are run on an identical, reproducible target route, and the difference is measured.

1.2 Design Philosophy

The *follower* deliberately uses none of the standard ROS2 autonomy stack: `slam_toolbox` for mapping, Nav2 for global planning and control, or `robot_localization` for state estimation. The omission is a design decision rather than a simplification.

The reason is that a map would not help *the follower*. A global planner answers the question “how do I get from here to a known goal pose while avoiding known obstacles?”. The follower does not have that question. Its goal is not a pose in a map: it is a moving object whose position is only known through the camera, from instant to instant. What the follower needs is: the *relative* position of the target, provided by the camera; knowledge of what is immediately in front of it, provided by the LiDAR; a short-term memory of where the target was, provided by odometry integrated over a few seconds. An occupancy grid adds no information to any of these three, at the cost of a substantial software stack to configure, tune and debug. The consequence is stated honestly in Section 11.5: without a map the follower cannot reason about corridors, rooms or exits, so once it has lost the target it can only rotate in place and wait for it to reappear.

The *target*, the scripted stimulus the follower chases and not the system under evaluation, is driven differently, by the official Nav2 navigation stack (AMCL for localisation, plus a planner and a controller), configured rather than reimplemented (Section 10): for the target’s own motion, an existing, complete navigation stack was available and is used as-is. This does not reintroduce

a map or a global planner into the *follower*, whose design remains exactly as described above. The target carries its own map and its own AMCL localisation for the sole purpose of following a fixed, reproducible route: it is a scripted actor with a capable script, not a change to what is being studied.

2 System Overview

2.1 Architecture

Four custom ROS2 nodes, each with one responsibility: the **detector** recognises the marker and publishes where the target is; the **controller** decides how the follower should move; **target_driver_nav2_node** feeds the target's route to Nav2, whose own stack then moves it along that route; the **logger** observes and records without ever intervening. Gazebo, the **ros_gz** bridge and the whole Nav2 stack run alongside these four but are not part of what this project writes.

The division into separate processes is not arbitrary. Because the nodes talk only through topics, the detector could be replaced by a learned one without a single line of the controller changing: the interface between them is a pose on a topic, not a function call. The same property makes the system easy to inspect: every link in the graph can be read from outside with **ros2 topic echo**, which is how most of the debugging in this project was actually done.

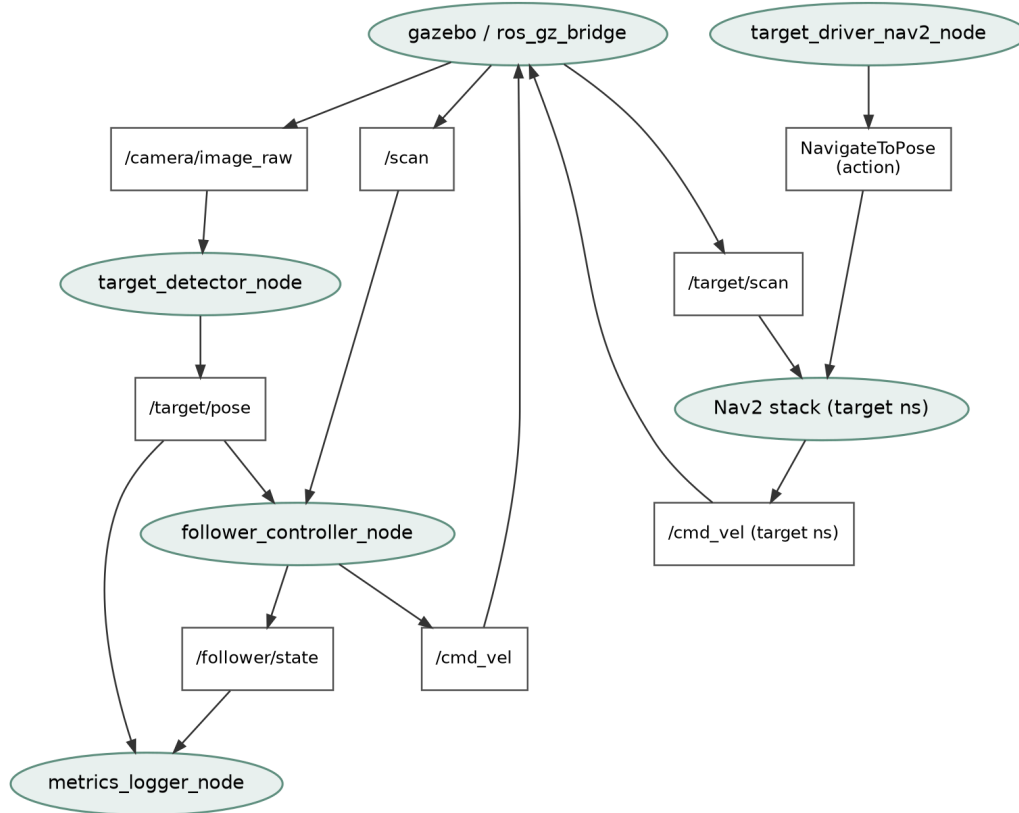


Figure 1: Node and topic graph, in the style of **rqt_graph**: ellipses are ROS2 nodes, boxes are topics (and the one action, **NavigateToPose**) sitting between a publisher and its subscriber(s).

2.2 Topic interface

Topic	Type	Meaning
/camera/image_raw	Image	follower camera, 10 Hz
/scan	LaserScan	follower LiDAR, 360°
/target/pose	PoseStamped	target position in odom, published only when the marker is seen
/cmd_vel	TwistStamped	velocity command to the follower
/follower/state	String	current state of the recovery machine

Loss of visibility is signalled by silence. The detector publishes nothing when it cannot see the marker, and the controller infers the loss from a timeout rather than from an explicit flag. One message type suffices and the semantics are unambiguous: every message on `/target/pose` is by construction a successful detection. A flag could also remain latched at its last value if the detector died; a timeout cannot.

2.3 Software environment

Component	Version / notes
Framework	ROS2 Jazzy
Simulator	Gazebo Harmonic, via <code>ros_gz_sim</code> / <code>ros_gz_bridge</code>
Vision	OpenCV, <code>cv2.aruco</code> , dictionary <code>DICT_4X4_50</code>
Robot platform	TurtleBot3 Burger (official ROBOTIS models)
Target navigation	Nav2 (official <code>navigation2</code> / <code>nav2_bringup</code> packages)
Containerisation	Docker, image based on <code>ros:jazzy</code>
Language	Python 3 (<code>rclpy</code> , <code>NumPy</code> , <code>OpenCV</code> , <code>pandas</code>)
Visualisation	RViz2

The entire workspace is containerised. The workspace is mounted as a volume rather than copied into the image, so that source changes require only a `colcon build` and not an image rebuild (the Docker image is described in the repository’s README).

3 Simulation Environment

3.1 The arena

The environment is a rectangle of 10×8 m enclosed by walls, with a chicane formed by two vertical blades: one hanging from the top edge, the other rising from the bottom. The free gaps therefore alternate, and any route crossing the arena is forced to weave. Three fixed ArUco panels are mounted flush against the walls. It was built for this project rather than taken from the official package, because the official arenas are dimensioned for one robot navigating alone.

The arena is generated by a short script, `generate_world.py`, from a geometry written once in `worlds.py`. The same geometry is used a second time, in `scripts/generate_map.py` (Section 10) and by the trajectory validation tests, to check that every trajectory and every map derived from it keeps a safe distance from the walls. Describing the geometry in two independent places is how a map and the simulated world can silently disagree; generating both from one function removes that possibility by construction.

3.2 The two robots

Both robots derive from the official ROBOTIS TurtleBot3 Burger models and differ in one respect each. The follower carries a replaced camera (Section 3.2.1); the target carries the ArUco panel (Section 3.2.2) and, since this project’s move to Nav2, its own 2D LiDAR (Section 10).

3.2.1 The camera was replaced

The follower does not carry the camera ROBOTIS ships. The switch was driven by simulation performance: the official `turtlebot3_burger_cam` runs a fisheye lens at 320×240 pixels and 30 Hz, and rendering a fisheye view in Gazebo is markedly more expensive than a standard camera at comparable resolution. The replacement is a pinhole camera at 480×360 , running at 10 Hz.

With the pinhole camera deployed, the practical detection range is estimated at roughly 1.8 m, based on the marker size needed for the ArUco decoder to resolve the pattern reliably. This figure is an engineering estimate embedded as a comment at the point in the code where it matters (`models/turtlebot3_follower/model.sdf`), not a value measured in a dedicated calibration experiment. It governs the difficulty of the whole scenario: it is what makes a following distance of 1.5 m a demanding choice rather than a comfortable one.

3.2.2 How the marker is mounted on the target

The panel is a textured square of 0.18 m placed behind the centre of the robot, with a yaw that turns its textured face backwards, towards the pursuing follower. The marker is on the rear, and that is the whole problem: mounting it there is the natural geometry of pursuit, but it is also the origin of the dominant loss mechanism of the entire project, described in Section 4.4. As soon as the target turns, the panel rotates away from the line of sight and becomes unreadable well before any obstacle interposes.

4 Perception: Marker Detection and Pose Estimation

4.1 The marker

An ArUco marker is a binary square: a thick black border around a 4×4 grid of cells encoding an identifier. The dictionary used, `DICT_4X4_50`, is not the set of all 2^{16} patterns: OpenCV generates it by maximising the minimum Hamming distance between any two of its fifty elements across all four rotations, a property that matters because the detector does not know in advance how the marker is oriented in the image. The practical consequence: a few misread cells do not produce a wrong identifier, they produce *no* identifier. That is the desirable way to fail. The target carries ID 3; the three fixed panels on the walls carry 0, 1 and 2.

4.2 From the image to the target position

Finding the marker in the image is only half the job: the detector returns the pixel coordinates of its four corners, and what the controller needs is how far away and in which direction the target is. Going from one to the other is the Perspective-n-Point problem, and this project does not implement it: OpenCV solves it in `solvePnP`, using the closed-form `IPPE_SQUARE` method for a planar square. What the project supplies is the four *object points* (the corners of the black square in the marker's own frame) and converts the result into the two quantities the control law uses: the distance d and the bearing β , the horizontal angle between the camera axis and the target.

The size passed to the solver is the black square, 0.15 m (`marker_size` in Section 4.3), not the 0.18 m of the white panel that carries it: the quiet zone is not part of what the solver reasons about.

4.3 The detector node

The node is adapted from ROBOTIS' official `turtlebot3_aruco_tracker` package (Apache License 2.0), rather than written from scratch: the full account of what is kept and what is changed

is in Section 12 and in the header comment of `target_detector_node.py` itself. Two interface decisions are worth stating here.

It publishes only when it sees. No message is emitted when the marker is absent, so the controller infers the loss from the silence of the topic through a timeout, rather than from an explicit flag that could remain latched at a stale value.

It filters by identifier. Four markers of the same dictionary exist in the arena, the three fixed panels and the target, and more than one is often visible in a single frame. A single check selects the one carrying ID 3, and the main trajectory deliberately passes alongside all three fixed panels so that this filter can be seen working. This filter has no equivalent in the original tracker, which was built to follow a single parking marker and reacts to whichever one it sees.

4.4 The dominant loss mechanism: incidence angle

The marker is mounted on the *rear* of the target, so it is readable only while the follower is looking at it more or less from behind. As soon as the target turns, the panel rotates away from the line of sight and the projected square degenerates until the decoder can no longer read it, well before any obstacle interposes itself. The consequence is that in this system the dominant cause of target loss is the **geometry of the turn**, not occlusion by obstacles: the frequency of loss correlates with the density of turns along the route. The single route defined in `trajectories.py`, `trajectory_1`, is designed around this: it weaves through the chicane with turns that make the marker go edge-on repeatedly, which is what makes the two recovery strategies actually distinguishable.

5 Reference Frames

5.1 The velocity must be estimated in a fixed frame

The same physical quantity, the position of the target, is handled in three frames: the *optical* frame, where `solvePnP` produces it; `base_footprint`, where the control law needs distance and bearing relative to where the robot is pointing now; and `odom`, a fixed frame. The detector transforms the pose into `odom` *before* publishing it, rather than leaving that to the controller. Estimating velocity by differentiating the position observed in the camera frame would differentiate a quantity that also moves because *the follower moves*; the result would be the velocity of the target relative to the follower, not its velocity, and since the follower is turning precisely in the moments before a loss, extrapolating that estimate would predict a position systematically displaced in the wrong direction.

5.2 Two things kept out of the transform tree on purpose

TF2 organises reference frames as a tree, and any node can ask it to convert a point from one frame to another. Two things are deliberately kept out of that tree.

The true pose from the simulator. Gazebo knows exactly where both robots are. That information is published on a topic (`/follower/ground_truth`, `/target/ground_truth`), but it is *not* inserted into `/tf`. If it were, a lookup somewhere in the controller could eventually resolve through it, and the follower would end up using knowledge it is not supposed to have.

The target's own frames. The target's DiffDrive plugin, and now also its Nav2 stack, produce frames named `odom` and `base_footprint`: the same names the follower uses. Publishing both on `/tf` would create two robots claiming the same names in the same tree, and every lookup would resolve non-deterministically. The target's entire TF tree, map frame included, is therefore routed to `/target/tf`, where nothing on the follower's side listens to it.

6 Pursuit Control

6.1 The control law

The TurtleBot3 Burger is a differential-drive platform: a linear velocity v and an angular velocity ω , and no sideways motion. The task is described by two errors:

d	distance to the target	measured by the camera
β	bearing to the target	measured by the camera, positive to the left
d^*	desired following distance	1.50 m, a parameter
$e_d = d - d^*$	distance error	positive when too far
$e_\beta = \beta$	bearing error	zero when the target is straight ahead

The law adopted is a pair of proportional loops, one per input:

$$v = (K_v e_d + v_{ff}) \cos(\min(|e_\beta|, \pi/2)), \quad \omega = K_\omega e_\beta, \quad (1)$$

where $v_{ff} = \hat{v}_t \cdot \hat{u}_{los}$ is the feed-forward term described below.

Parameter	Value	Rationale
K_v (k_linear)	0.60	gain of the distance loop
K_ω (k_angular)	1.50	gain of the bearing loop; higher values oscillate
d^* (desired_distance)	1.50 m	against a ≈ 1.8 m detection range
Dead bands	0.10 m, 0.05 rad	above the measurement noise floor

6.2 Where the law comes from

The structure of this law is not invented for this project: it is the standard *Cartesian regulation* approach for a unicycle, which drives it to a target position by a steering command proportional to the angular error and a linear command proportional to the projection of the position error onto the robot's own heading axis:

$$\omega = k_2(\text{atan2}(y, x) + \pi - \theta), \quad v = -k_1(x \cos \theta + y \sin \theta). \quad (2)$$

With the position error expressed in polar form (magnitude d (distance) and angle β (bearing) with respect to the robot's heading) the projection term $x \cos \theta + y \sin \theta$ is exactly $d \cos \beta$, and this becomes $v = K_v d \cos \beta$ and $\omega = K_\omega \beta$: the same two proportional loops, one per input, coupled only through the same cosine projection.

Two things are added on top of that formulation, both necessary because this is *pursuit* of a moving target at a standoff distance, not *regulation* to a fixed point at the origin. First, the error is taken with respect to a non-zero desired distance d^* rather than zero ($e_d = d - d^*$), since the goal is to hold station behind the target, not to reach it. Second, the feed-forward term v_{ff} is added because a purely proportional law regulating to a *moving* reference has a steady-state lag (quantified in the next subsection), which plain regulation to a fixed goal does not exhibit and therefore has no need to correct for.

6.3 What each term buys

The cosine factor is the only coupling between the two loops, and it encodes the non-holonomic constraint: if the target is 90° to one side, moving forward does not reduce the distance at all, and the robot would describe an arc and get further away first. The cosine suppresses translation while the robot is badly aligned, so it turns first and drives second.

Dead bands on distance and bearing sit just above the measurement noise, so that noise alone does not produce a continuous alternation of small forward and backward commands.

Feed-forward on the target velocity is the term that repays explanation, because its absence is not obvious. A proportional loop tracking a moving reference always lags behind it: to hold station behind a target moving at v_t the follower must itself move at v_t , and a purely proportional law can only produce that command from a non-zero error, so the follower would settle further back than intended, close enough to the edge of the marker's detection range to lose it on a straight line alone. Feeding the already-estimated target velocity forward removes that lag directly, instead of adding an integral term and the anti-windup logic it would need.

Reversing is allowed only while the target is getting closer. If the follower ends up nearer than the setpoint but the target has started moving away, backing off would open the gap at the sum of both speeds, pushing the marker out of detection range faster than simply holding position would.

The setpoint ramps after a re-acquisition. Regaining contact at close range leaves a large negative error, and demanding the full setpoint at once would sit the follower still until the target receded past it. The setpoint instead starts at the distance where contact was regained and rises linearly to nominal over a few seconds: right after finding the target again, the priority is not to lose it, not to restore the geometry immediately.

6.4 Gain tuning

The gains were tuned following the standard sequence for cascaded loops: the angular loop in isolation first, then the linear loop, then the interaction, each pushed as fast as it would go without visible oscillation.

7 Motion Prediction

When the marker disappears, the controller needs an answer to a simple question: *where is the target now?* It cannot see it, but it saw it a moment ago, and it can work out how fast it was going. This section, together with Section 8, is the part of the project left entirely original, with no external attribution.

7.1 Estimating how fast the target is moving

The detector supplies a position roughly ten times a second. The controller keeps a short window of recent positions and works out the target's velocity by fitting a straight line through them, rather than simply subtracting the last two positions and dividing by the time between them: differencing two noisy points amplifies the noise instead of averaging it out. A minimum number of samples is required before any estimate is trusted, and the result is capped at the platform's own top speed, since a fit built from a short, noisy window can occasionally suggest a speed the robot could not physically have.

7.2 Predicting where it went

Once the speed is known, the prediction is deliberately plain: the target is assumed to carry on in a straight line at the speed it had. A more elaborate model would need an extra quantity (acceleration, or rate of turn) estimated from the same short, noisy window, and the rate of turn in particular becomes unmeasurable at the exact moment the target goes out of view, which is when it would be needed. A minimum-speed threshold prevents the residual noise on a stationary target from being extrapolated into drift.

7.3 The same code serves both pursuit and prediction

The FOLLOW and PREDICT states share the same code path (`measurement()` in `follower_controller_node.py`): the target's position is *always* obtained by extrapolation. While the marker is visible the elapsed

interval is a few hundredths of a second and the correction is negligible; when it disappears the interval grows and the extrapolation becomes the only information available. The benefit is practical: a defect in the extrapolation shows up during ordinary pursuit, not only in the rare moments when the target vanishes.

7.4 The prediction horizon

The horizon is how long the follower keeps trusting the estimate after the marker disappears. It is chosen to comfortably cover the time a typical turn takes at the target's speed, so the prediction is not abandoned just before it would have paid off, while staying short enough that the ground covered on an assumption that is, by hypothesis, becoming false stays small.

The horizon is not the only way out: it is fixed and cannot adapt to how sharp the current turn actually is, so the prediction is also abandoned on a second, independent criterion described next.

8 The Recovery State Machine

8.1 States and transitions

The follower's behaviour is governed by a finite state machine with three states, FOLLOW, PREDICT and SEARCH:

State	Behaviour
FOLLOW	Nominal pursuit; the control law of Section 6 is active.
PREDICT	The marker has been lost for more than <code>lost_timeout</code> (0.40 s). The follower continues the pursuit on the extrapolated position, at 70% of the nominal linear velocity: moving more slowly while blind is deliberate, since the estimate degrades with time and 70% leaves more margin for the safety arbitration to react if the estimated position lies beyond an obstacle.
SEARCH	The follower rotates in place, scanning towards the side the target was last seen on, until the marker reappears.

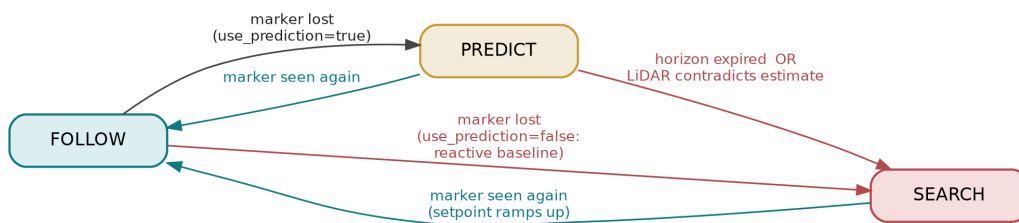


Figure 2: The three states and their transitions. Red arrows escalate towards the blind search; teal arrows recover; the reactive baseline (Section 11) simply disables the FOLLOW→PREDICT transition, so every loss goes straight to SEARCH.

Loss of visibility is detected by timeout (`lost_timeout`= 0.40 s, four missed camera frames at 10 Hz) rather than by an explicit flag, consistently with the interface choice of Section 4.3.

8.2 Abandoning the prediction on a sensor contradiction

The prediction is abandoned on two criteria, not one. The first is the horizon. The second is geometric: if the LiDAR reports an obstacle within braking distance in the direction of the estimated position, and the estimate lies well beyond it (`blocked_towards()`: `free < d_safe`

AND `free < distance - blocked_margin`), the follower is *pinned* and the estimate cannot be reached.

Both conditions are required, and that is the point: a rule that aborted the prediction whenever *any* obstacle lay in that direction would not work here. In the chicane the target is lost precisely because the panel goes edge-on behind one of the blades, so an obstacle in that direction is the *normal* case, not the exception; with only that condition, almost every prediction would be aborted within a fraction of a second of the loss, and the predictive strategy would stop differing from the reactive one.

8.3 The two strategies under comparison

The whole experiment rests on comparing two behaviours that differ in *one thing only*. In the code that is a single boolean, `use_prediction`, checked in exactly one place (`on_target_lost()`):

```

1 def on_target_lost(self):
2     # The ONLY point where the two strategies diverge.
3     if not self.use_pred:
4         self.enter_search('target lost -> SEARCH (reactive baseline)')
5     elif len(self.history) >= 3:
6         self.lost_since = self.now()
7         self.set_state(PREDICT, 'target lost -> PREDICT')
8     else:
9         self.enter_search('target lost -> SEARCH (too few samples)')
```

Listing 1: The only point where the two strategies diverge (`follower_controller_node.py`).

With `use_prediction: false` the PREDICT state is skipped and a loss leads directly to the search. Everything else is identical: the same control law, the same gains, the same safety arbitration, the same target route. The single independent variable is the blind pursuit stage.

9 Safety Arbitration

9.1 The conflict

The project proposal identifies this problem directly:

The visual objective and the safety constraint may require opposite commands, for example when the target is located beyond an obstacle. The arbitration logic must resolve the conflict without producing deadlocks or losing the target permanently.

A naive priority scheme (safety always wins) produces a robot that freezes in front of every obstacle and loses the target. A naive opposite scheme produces a robot that collides.

9.2 Provenance of this section's code

`arbitrate()` and its helpers are adapted from ROBOTIS' own `turtlebot3_automatic_parking` package (Apache License 2.0, the same package family the detector is adapted from). That reference's final parking step reads a fixed range of scan indices (`ranges[150:210]`, a 60-sample rear sector centred at 180°) and checks the minimum against a single distance threshold to decide whether to keep reversing or stop: a binary go/stop response, with no scaling and no notion of which side is actually free. What is kept from it is the pattern itself: reading a scan sector by a fixed range of *array indices* rather than computing an angle per sample, which is exact here only because this LiDAR is known to always return 360 samples, one per degree, starting at zero (`sector_min()` in `follower_controller_node.py`, and the full comment preceding `arbitrate()` for the exact reference).

Everything past that single threshold-and-stop check is this project's own addition, because on its own the reference is exactly the failure mode this project's design argues against: a binary

response with no scaling and no notion of which side is free. What is added: three graded regimes instead of a single threshold (free / braking / stop, scaled linearly rather than switched), a scan of both sides of the front sector to always turn towards whichever is freer, a guaranteed way out even at zero commanded speed, and the same treatment applied symmetrically to the rear sector for reverse motion.

9.3 How the LiDAR limits the visual command

The rule is simple to state: **the LiDAR never replaces the visual command, it only limits how fast the robot may advance**, within $\pm 30^\circ$ (`front_sector_deg / 2`) of straight ahead.

Regime	When	What happens
free	$d \geq 0.35$ m	the command passes through unchanged
braking	$0.22 < d < 0.35$ m	forward speed scaled linearly; a turn towards the freer side is progressively added
stop	$d \leq 0.22$ m	forward motion cancelled, only the escape turn remains

Rotation is never suppressed, so the follower can keep turning to hold the target in frame even while braking: this is what stops the safety layer from causing the very loss it exists to avoid. **There is always a way out**: as braking increases, a rotation towards the freer side is added even at zero commanded speed, otherwise standing still in front of a wall would be a stable state with no exit. The rear sector is arbitrated by the identical law, since the Burger's LiDAR spans a full 360° ; reverse motion is capped at half the forward speed (`reverse_factor=0.5`), because opening a gap is never urgent.

10 Experimental Infrastructure

10.1 The target as a reproducible stimulus

The target is not part of the system under test. It is the stimulus, and its required property is *reproducibility*: the comparison between the two strategies is meaningful only if both are exposed to exactly the same scenario, a requirement the proposal states explicitly. What drives the target towards that requirement is the official Nav2 navigation stack.

10.2 Why Nav2 drives the target

For the target's own motion, follow a fixed sequence of waypoints while avoiding the arena's obstacles, a complete, official, actively maintained navigation stack already exists and needed no reimplementations. Driving the target with it, rather than with a bespoke proportional controller, means the planner, the controller, the localisation and the whole behaviour tree are the unmodified upstream code, only configured.

This does not reintroduce a map, SLAM or a global planner into the *follower*. The follower's own design (Section 6) is unchanged: it still has no knowledge of the arena, no localisation of its own, and no access to the target's map or TF tree (Section 5.2, restated below for the target's Nav2 stack specifically).

10.3 What is official Nav2 and what is this project's own

Nothing of Nav2 itself is rewritten. `launch/target_nav2.launch.py` does four things:

1. loads the official `nav2_params.yaml` shipped with `nav2_bringup` and applies on top of it only the parameter deltas in `config/nav2_overrides.yaml`, about thirty keys, not a copy

of the file (the companion study guide gives every one of them individually, key by key); the base file therefore always matches whatever Nav2 version is installed, and every change this project makes is visible in one short file;

2. publishes the one static transform the target's model does not provide on its own (`target/base_footprint` → `target/base_scan`), following exactly the same namespacing convention `nav2_bringup` itself uses for every one of its own nodes;
3. includes the official `nav2_bringup/launch/bringup_launch.py` (`map_server`, `AMCL`, `planner_server`, `controller_server`, `behavior_server`, `bt_navigator`, the lifecycle manager), in the `/target` namespace, with a map generated by `scripts/generate_map.py`;
4. starts `target_driver_nav2_node`, adapted from `nav2_simple_commander`'s own security-patrol demonstration (`demo_security.py`, Apache License 2.0): about twenty lines changed out of roughly 110, to read the route from `trajectories.py` instead of a hard-coded list, start from the waypoint nearest the spawn pose, orient each pose along the next leg instead of all facing $+x$, and repeat the closed loop instead of reversing it.

The map. `scripts/generate_map.py` produces the static occupancy grid `AMCL` localises against from exactly the same geometry function used for the world file and the trajectory tests, `worlds.clearance()`, a third consumer of that single source of truth. The three wall-mounted `ArUco` panels are deliberately *not* marked as obstacles in this map: they are mounted 0.3–0.5 m above the ground, above the plane the LiDAR scans, so the real sensor never sees them and the map used for localisation must not expect them either.

The sensor. The target's model has no LiDAR of its own by default: nothing in the original `ROBOTIS TurtleBot3` target model provides one. The block added to `models/turtlebot3_target/model.sdf` is the same `hls_lfcd_lds` sensor already present on the follower, unchanged apart from its topic and frame names.

Everything Nav2-related lives under /target, TF included (`/target/tf`, never `/tf`), for the same reason given in Section 5.2: the follower's own transform tree must never be able to resolve a lookup through the target's map frame or localisation.

10.4 Two parameters worth naming explicitly

Two values in `config/nav2_overrides.yaml` were not obvious defaults and were tuned against this specific arena.

Costmap inflation, reduced from Nav2's default to `inflation_radius= 0.35` m. The chicane's gap is barely a metre wide; at a larger inflation radius the planner would treat the gap as effectively sealed and refuse to route the target through it at all.

Planner tolerance on intermediate waypoints, tightened from the Nav2 default of 0.5 m to `tolerance= 0.15` m in `planner_server`. Several waypoints in `trajectory_1` are small, deliberate detours: as little as 0.6 m off the main axis, written to bring the target close to one of the fixed markers. At the default tolerance the planner is free to satisfy such a waypoint from half a metre away, which is enough to cut the detour altogether; visited too loosely, the target would stop passing near the marker the route was designed to pass near. This was found empirically, by comparing the planned route against the one Nav2 actually drives.

10.5 Two turns deliberately placed away from any obstacle

Nav2's own obstacle avoidance slows the target down near a wall or the chicane, regardless of anything the follower does, which changes what a following robot that has stopped is penalised for (Section 11.3.2 discusses this in full). Of `trajectory_1`'s fourteen turns, ten sit close enough to a wall or a chicane blade for this to apply; four, at (0.0, 2.0), (0.0, -2.0), (2.0, 1.0) and (-2.0, -1.0),

are more than 1.5 m from any obstacle, comfortably outside the 0.35 m inflation radius, so the target passes through them at its normal commanded speed rather than a damped one. The incidence-angle loss mechanism of Section 4.4 does not care whether an obstacle is nearby, only whether the target is turning, so these four turns still produce marker losses; what differs is that the target does not slow down for them. This is not a change to the arena (`worlds.py`, and the map and world files generated from it, needed no change, so neither Nav2’s costmap tuning nor the geometric test suite did either): it is a property of *where*, along the route, the waypoints happen to sit.

10.6 Ground truth for the target, and why it is legitimate

The distinction between the target and the object of study holds under Nav2 exactly as it does for the follower. AMCL’s localisation of the target is not a shortcut the follower benefits from: the follower perceives the target exclusively through its own camera and LiDAR, and the target’s map, its AMCL pose and its whole Nav2 stack are confined to `/target`. The mechanism by which the scripted actor reaches its waypoints has no bearing on the follower’s own task.

11 Testing and Validation

11.1 Experimental protocol

Independent variable. The strategy, and nothing else. Same control law, same gains, same safety arbitration, same target route: the two configurations differ in one boolean (`use_prediction`).

Four runs per configuration, 120 s of simulated time each. Repeating the same configuration matters because the simulation is not perfectly repeatable: sensor noise and thread scheduling make two identical runs come out slightly differently. Four runs allow the spread *within* a configuration to be compared against the difference *between* the two.

Two precautions make the runs comparable. Each run is stopped by the logging node when the *simulated* clock, not the wall clock, has advanced by 120 s: Gazebo does not run in step with real time, and a wall-clock timer would give the two strategies different amounts of the quantity that actually matters. And the two strategies are **alternated** rather than run in blocks, because Gazebo leaves processes behind that measurably degrade the following run; run in blocks, whichever strategy went second would inherit all of that degradation.

```
1 ./scripts/run_experiments.sh
```

Listing 2: The full campaign (`scripts/run_experiments.sh`).

11.2 Metrics

Metric	Meaning
<code>tracking_loss_events</code>	times the marker became invisible
<code>definitive_losses</code>	of those, how many escalated to the blind search: the quantity the prediction is expected to reduce
<code>mean_recovery_time_s</code>	mean interval between a loss and re-acquisition
<code>rmse_distance_m</code>	RMSE (root-mean-square error) of $d - d^*$, defined only while the target is visible: with no marker there is no measurement, and inventing one would flatter whichever strategy loses the target more often
<code>collisions</code>	samples with an obstacle below 0.16 m

The accuracy is measured against what the camera sees, not against a reconstructed true position: the ArUco pose estimate is the quantity the control loop actually acts on, and it is available exactly when the follower is doing its job.

11.3 Results

Table 1 reports the campaign with the target driven by Nav2, four runs per configuration, 120 s of simulated time each, on `trajectory_1`.

Table 1: Reactive versus predictive strategy, target driven by Nav2. Mean $\pm$ standard deviation over four runs.

Metric	Reactive	Predictive	Change
<code>duration_s</code>	120.00 ± 0.00	120.00 ± 0.00	–
<code>tracking_loss_events</code>	6.75 ± 1.50	7.00 ± 0.82	+4%
<code>definitive_losses</code>	6.75 ± 1.50	3.00 ± 0.00	–56%
<code>mean_recovery_time_s</code>	7.09 ± 1.70	4.96 ± 0.37	–30%
<code>rmse_distance_m</code>	0.145 ± 0.006	0.181 ± 0.010	+25%
<code>collisions</code>	0	0	–

11.3.1 What the comparison shows

The prediction does not reduce how often the target becomes invisible by much (that is set mostly by the geometry of the route, the same fourteen turns every lap) and this campaign confirms it again: `tracking_loss_events` differs by only 4%, with the predictive strategy if anything losing the marker very slightly *more* often. What changes is what happens *next*. Summed over the four runs of each configuration:

	losses	of which definitive
Reactive	27	27 (100%)
Predictive	28	12 (43%)

The reactive baseline escalates to the blind search on *every single loss* it suffers, across all four runs. The predictive strategy resolves 16 of its 28 losses within the three-second horizon, before the recovery behaviour is needed at all. This is the result the whole recovery design (Sections 7–8) exists to produce, and it is the most stable signal in this campaign.

11.3.2 An honest reading of the RMSE figure

Three of the four metrics in Table 1 agree with each other and with the proposal’s expectation: the predictive strategy has fewer definitive losses, and also a faster mean recovery. The reason both figures point the same way is the route (Section 10.5): four of `trajectory_1`’s fourteen turns sit more than 1.5 m from any obstacle, comfortably outside Nav2’s 0.35 m inflation radius, so at least some losses happen while the target is moving at its normal, undamped speed rather than one Nav2 has itself slowed down near a wall or the chicane. A follower that has stopped is not spared by the target slowing down for reasons of its own on every turn.

The RMSE figure alone still points the other way: the reactive strategy’s average distance error is lower than the predictive one’s. A plausible, not yet isolated, explanation is that RMSE is computed against the fixed setpoint $d^* = 1.5$ m regardless of the five-second ramp that deliberately holds the setpoint below d^* right after a re-acquisition (Section 6): the predictive strategy

re-acquires the target slightly *more* often per run than the reactive one does (30 versus 27 re-entries into FOLLOW, summed over the four runs), so a larger share of its FOLLOW-state samples fall inside a ramp window, where the controller is intentionally not yet at d^* . This would inflate the predictive strategy’s RMSE for a reason unrelated to tracking quality, but it has not been confirmed by a dedicated experiment (e.g. excluding ramp windows from the RMSE calculation) and is reported here as a hypothesis, not a finding.

The metric this project’s design was built to move (**definitive_losses**) is unaffected by either explanation and remains the strongest, most stable signal in this campaign. RMSE and mean recovery time are reported honestly rather than omitted, but they should be read as sensitive to the specific dynamics of whatever drives the target, while **definitive_losses** measures the property the recovery logic was actually designed around: whether a loss of the marker is recoverable at all.

11.4 Summary of the work

A complete leader–follower navigation system was developed and evaluated in simulation on the TurtleBot3 platform with ROS2 and Gazebo. The follower tracks a target identified by an ArUco marker using onboard vision alone, holds a prescribed following distance through two decoupled proportional loops with feed-forward of the estimated target velocity, and avoids obstacles through a graded arbitration between the visual objective and the LiDAR constraint. The distinguishing contribution is the recovery of the target after a loss of visibility, structured in two stages: a bounded blind pursuit on a constant-velocity extrapolation, abandoned either on a fixed horizon or when the LiDAR contradicts the estimate; and, if that fails, a rotation in place until the marker reappears. The design was evaluated by controlled comparison against the purely reactive baseline defined in the proposal. Over four runs per configuration, the reactive baseline escalated every one of its 27 losses of visibility to the blind search, while the predictive strategy resolved 16 of its 28 within the three-second horizon: a 56% reduction in definitive losses, a 30% faster mean recovery, and no collisions in either configuration.

11.5 Limitations

The search has a structural ceiling. A genuinely informed search would require the follower to have a map; without one, once the target is lost for longer than the prediction horizon the follower can only rotate in place and wait. This is a consequence of the design decision of Section 2.2 and is accepted as such.

The motion model is first order. Constant-velocity extrapolation degrades if the target turns while invisible, precisely the case in which it was lost. The bounded horizon of Section 7.4 is what keeps this from mattering much in practice.

RMSE and mean recovery time are sensitive to the target’s own dynamics, as Section 11.3.2 discusses: they should be read alongside, not instead of, **definitive_losses**.

Single target, simulation only. The system tracks one identifier; multi-target tracking would require data association, a different problem. All results are obtained in Gazebo; transfer to hardware would require camera calibration and would face harsher lighting, motion blur and a different LiDAR noise model.

11.6 Future work

- **Follow the route, not the target.** The follower steers at where the target *is*, so it cuts corners by an amount that grows with the following distance. Steering at a point already driven by the target would remove this.
- **Give the search a map.** Even a rough occupancy grid, built during the pursuit rather than supplied in advance, would let the follower reason about where a lost target can

plausibly have gone, without reopening the perception-only design of the follower itself.

- **Estimate the uncertainty, not just the position.** A Kalman filter in place of the sliding-window regression would report how confident the estimate is, letting the horizon adapt instead of being fixed at three seconds.
- **Exclude the re-acquisition ramp from the RMSE calculation.** The hypothesis in Section 11.3.2 for why RMSE still favours the reactive strategy (more frequent re-entries into FOLLOW for the predictive strategy, each opening a five-second window where the set-point is deliberately below d^*) is mechanistically plausible and grounded in the actual counts from this campaign, but has not been isolated by a dedicated experiment. Logging whether each sample falls inside a post-re-acquisition ramp, and computing RMSE with and without those samples, would turn a plausible explanation into a demonstrated one.

12 Provenance of the Code

Table 2 summarises every substantial external source this project draws on, with the target’s motion prediction and recovery logic left original; the full account of what is kept and what is changed, file by file, is in the header comment of each file named below.

Table 2: External sources this project’s code is adapted from.

File	Source	Licence	What is kept
target_detector_node.py	ROBOTIS turtlebot3_aruco_tracker	Apache-2.0	node shape, detection loop
follower_controller_node.py, arbitrate()	ROBOTIS turtlebot3_automatic_parking	Apache-2.0	index-based scan-sector reading (§9)
target_driver_nav2_node.py, and Nav2 itself	nav2_simple_commander’s demo; nav2_bringup; navigation2	Apache-2.0	patrol-demo client pattern; the navigation stack, unmodified
metrics_logger_node.py, install_shutdown_logging()	evannsmc/ROS2Logger	MIT	signal/atexit run-once shutdown pattern

Table 3: Theoretical basis this project’s code implements (not a code repository: a standard control-theory result, cited for the same reason a textbook derivation would be).

File	Source	Licence	What is kept
follower_controller_node.py, visual_command()	Cartesian/polar regulation for a unicycle (standard control theory)	textbook	the two-loop structure (§6.2)

Left deliberately original, with no claimed external source: the motion prediction and the recovery state machine (Sections 7–8), which the course names as the explicit exception; the extensions to the pursuit control law that go beyond plain regulation-to-a-point (the non-zero standoff distance, the feed-forward term, the dead bands, the range-rate gating on reversing, and the post-reacquisition setpoint ramp, §6.3), for which the theory above gives the base structure but not these additions; and a small amount of supporting infrastructure, the arena and trajectory data (`worlds.py`, `trajectories.py`), the world and map generators (`generate_world.py`, `generate_map.py`), and the results-analysis script (`analyze_results.py`), for which no comparably substantial external source was found.

13 Repository

The complete source code, the Docker environment, the test suite and the analysis scripts are available at:

<https://github.com/luigipioficco/turtlebot3-leader-follower>

A companion document, intended for the author's own study rather than for submission, walks through every file in the repository (starting from the **Dockerfile**) line by line.